

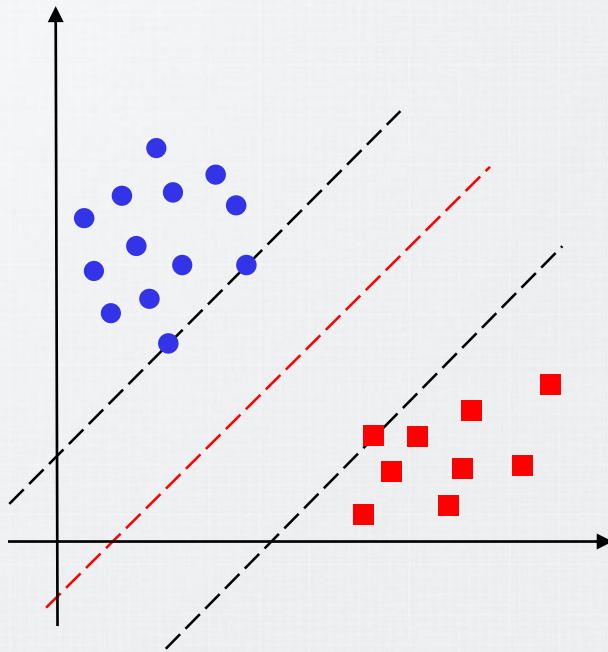
Support Vector Machines

Prof. Artur Jordão

Introduction

Support Vector Machines

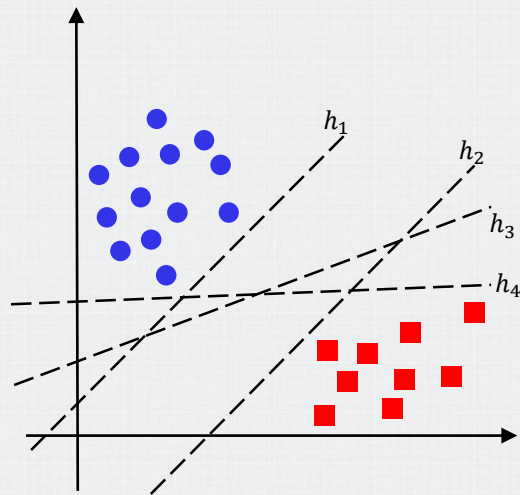
- The Support Vector Machine (SVM) model class was the most popular approach for *off-the-shelf* supervised learning
 - When we don't have any specialized prior knowledge about a domain



Intuition

Support Vector Machines

- Given that there are many possible linear classifiers (hyperplanes), which of the infinite hyperplanes should we choose?
- The key insight of SVMs is that some training examples are more informative than others, and the algorithm exploits this fact to determine the separating hyperplane
 - Paying attention to them can lead to better generalization



Properties

Support Vector Machines

- SVMs construct a maximum margin separator
- SVMs create a linear separating hyperplane
 - They have the ability to embed the data into a higher-dimensional space, using the so-called **kernel trick**
- SVMs (non primal) are nonparametric
 - A set of example points defines the separating hyperplane, not by a collection of parameter values
 - SVM model **keeps only the samples** that are closest to the separating plane

Preliminaries

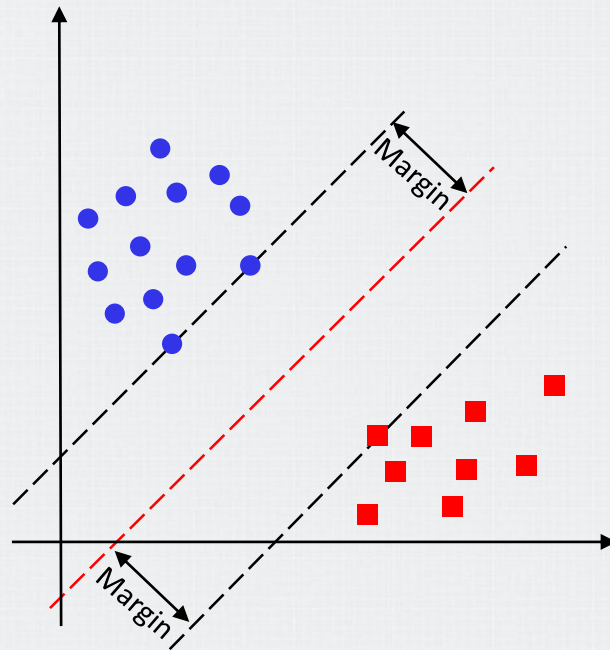
Support Vector Machines

- Let $w \in \mathbb{R}^m$ be a hyperplane and b a real number
 - While we could put the intercept into the weight vector w , SVMs keep the intercept as a separate parameter, b
- Given w and b , the SVM estimates the label in terms of $y = \text{sign}(w \cdot x + b)$
 - (Canonical) SVMs use the convention that class labels are $+1$ and -1
 - $y \in \{+1, -1\}$

Preliminaries

Support Vector Machines

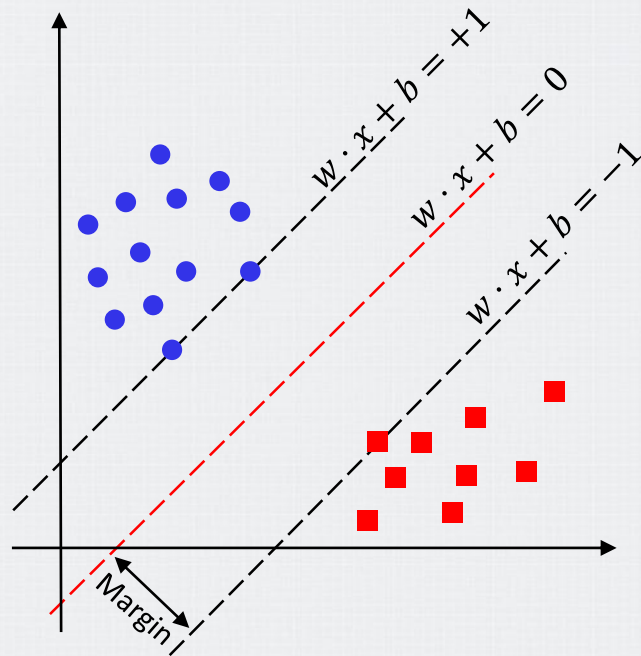
- The margin is the distance between the hyperplane and the closest training samples on either side
 - A large margin contributes to a better generalization
 - Therefore, we want to **maximize** the margin



Preliminaries

Support Vector Machines

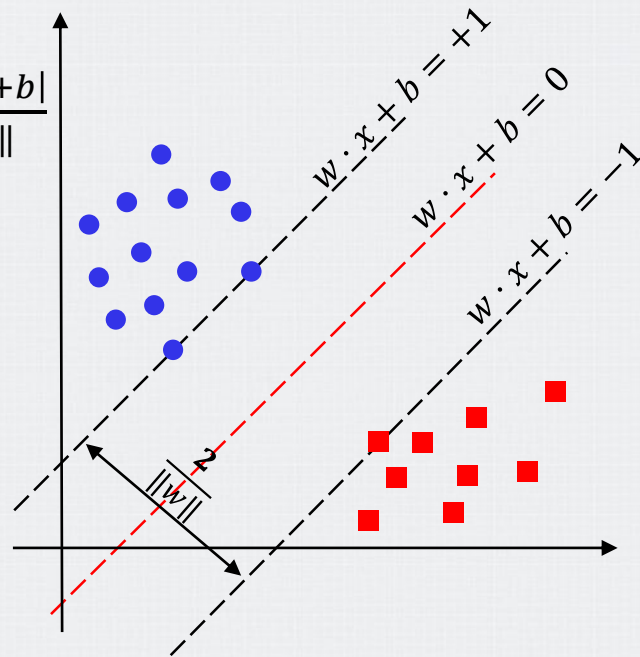
- The separating hyperplane consists of all points x that satisfy $\{x : w \cdot x + b = 0\}$
 - Decision boundary
- $w \cdot x + b = +1$
 - Margin of the positive class (+1)
- $w \cdot x + b = -1$
 - Margin of the negative class (-1)



Preliminaries

Support Vector Machines

- Geometrically speaking, the equations $w \cdot x + b = +1$ and $w \cdot x + b = -1$ define two parallel hyperplanes
 - Support vectors lie on the margin hyperplanes**
- Given that the distance from a sample to w is $\frac{|w \cdot x + b|}{\|w\|}$
 - When $w \cdot x + b = +1 \Rightarrow \frac{|1|}{\|w\|} = \frac{1}{\|w\|}$
 - When $w \cdot x + b = -1 \Rightarrow \frac{|-1|}{\|w\|} = \frac{1}{\|w\|}$
 - Therefore, the distance between these hyperplanes is given by $\frac{2}{\|w\|}$**



Preliminaries

Support Vector Machines

- $w \cdot x + b > 0$
 - x belongs to class $+1$
- $w \cdot x + b < 0$
 - x belongs to class -1
- $\frac{|w \cdot x + b|}{||w||}$
 - Distance from the hyperplane

Hands-on



Problem Definition

Support Vector Machines

- The distance between the hyperplanes is $\frac{1}{\|w\|}$, so the smaller the norm $\|w\|$, the **larger the distance between them**
 - Remember that the SVM seeks the separating hyperplane with **maximum margin**
- We can rewrite $w \cdot x + b \geq 1$ for $y = +1$ and $w \cdot x + b \leq -1$ for $y = -1$ as
 - $y(w \cdot x + b) \geq +1$
 - Note that $(w \cdot x + b) \leq -1 \Rightarrow -(w \cdot x + b) \geq +1$

Problem Definition

Support Vector Machines

- Given the previous definitions, the SVMs constraints are naturally
 - $\max \frac{1}{\|w\|}$ subject to $y_i(w \cdot x_i + b) \geq +1, \forall x_i \in X$
 - Since maximizing $\frac{1}{\|w\|}$ is equivalent to minimizing $\frac{1}{2} \|w\|^2$, in the separable case, SVM solves the following convex optimization problem

$$\min \frac{1}{2} \|w\|^2 \text{ subject to } y_i(w \cdot x_i + b) \geq 1, \forall x_i \in X$$

Quadratic Programming

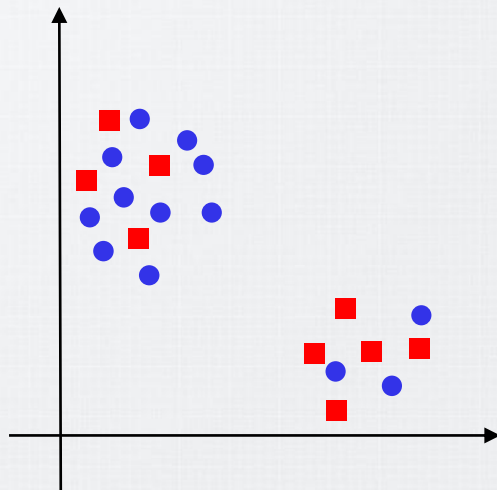
Support Vector Machines

- $\min \frac{1}{2} \|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1, \forall x_i \in X$ is an instance of standard quadratic optimization problem
 - The complexity depends on feature dimension
- A variety of commercial and open-source solvers are available for solving convex QP problems
- Lagrange multipliers (α)
 - $$L = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i (y_i (w \cdot x_i + b) - 1)$$
 - We minimize w.r.t. primal variables w and b (**SVM primal**)
 - The saddle point gives the solution
 - Saddle point conditions: $\frac{\partial}{\partial w} L(w, b, \alpha) = 0, \frac{\partial}{\partial b} L(w, b, \alpha) = 0$

Non-Separable Case

Support Vector Machines

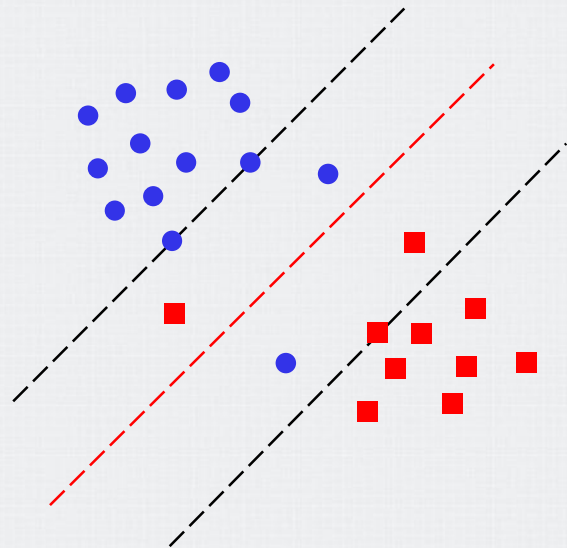
- If the data is not linearly separable, the previous algorithm will not work
 - If the two classes **are not linearly separable** such that there is **no** hyperplane to separate them
 - The two classes are not separable by a hyperplane
 - In most practical settings, the training data is not linearly separable



Soft Margin Hyperplane

Support Vector Machines

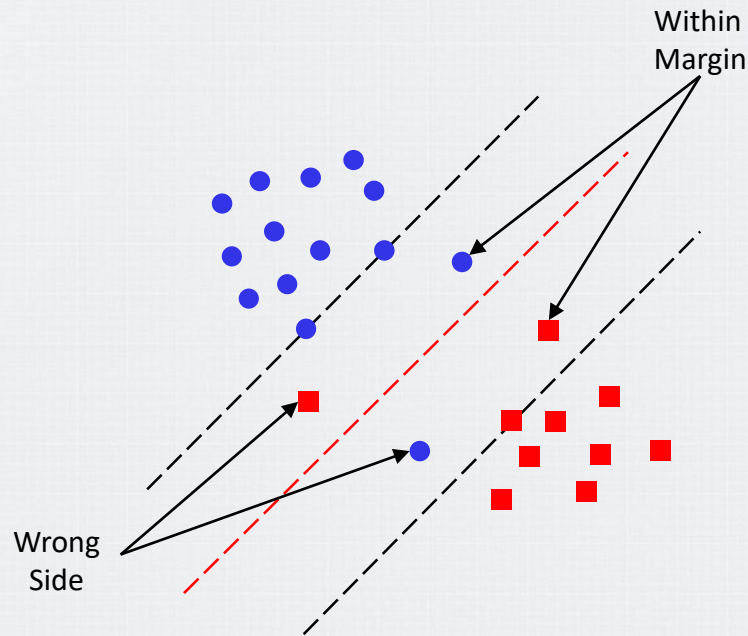
- When the data is **not linearly separable**, we instead allow some samples to be on the incorrect side of the region of maximum margin
 - Or even the incorrect side of the hyperplane
- The margin is soft because some of the training samples can violate it



Soft Margin Hyperplane

Support Vector Machines

- For the non-separable case, we introduce slack variables ξ_i to relax the constraints of the canonical hyperplane equation
- There are two types of deviation
 - A sample lies on the wrong side of the hyperplane (misclassified)
 - A sample lies on the correct side but within the margin



Soft Margin Hyperplane

Support Vector Machines

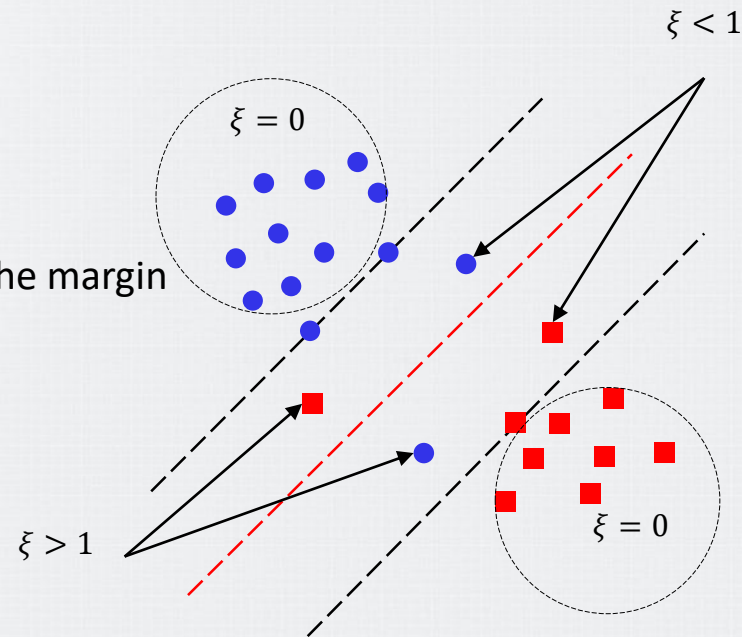
- To address the previous problem, we relax the original optimization problem to

- $y_i(w \cdot x_i + b) \geq 1 - \xi_i$

- $\xi_i = 0$
 - No problem (its is the original optimization)

- $0 < \xi_i \leq 1$
 - The model correctly classifies x_i but inside the margin

- $\xi_i > 1$
 - The model misclassifies x_i



Soft Margin Hyperplane

Support Vector Machines

- The soft error becomes $\sum_i \xi_i$
- Adding $\sum_i \xi_i$ to the SVM constraints, we obtain

$$\min \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \text{ subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \forall x_i \in X$$

- This is the standard SVM of popular machine learning libraries
- $|\{\xi^i > 1\}|$
 - An upper bound of the number of misclassifications
- $|\{\xi^i > 0\}|$
 - An upper bound of the number of margin violations

Soft Margin Hyperplane

Support Vector Machines

- C is the penalty factor
 - A hyperparameter we tune
- When C is small, the model allows **more margin violations**
 - The margin tends to be **wider**
- When C is larger, the model **penalizes violations more heavily**
 - The margin tends to be narrower
- In the limit $C \rightarrow \infty$, we recover the earlier SVM for separable data

[Hands-on](#)



Final Notes

Support Vector Machines

- After estimating w and b , the prediction for a given sample x_i
 - $\text{sign}(w \cdot x_i + b)$
- Therefore, what is the role of support vectors ($\{x : w \cdot x + b \pm 1\}$) during prediction?

Kernel Trick

Motivation

Kernel Trick

- When the problem is nonlinear, instead of trying to fit a nonlinear model, we can map the problem to a **new space** (**nonlinear** transformation – $\phi(\cdot)$) and then use a **linear model** in this new space
 - The linear model in the new space corresponds to a nonlinear model in the original space

[Hands-on](#)



Problem Definition

Kernel Trick

- The problem is the same as in the linear SVM, except that now we define the constraints in the new feature space ($\phi(\cdot)$)

$$\min \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \text{ subject to } y_i(w \cdot \phi(x_i) + b) \geq 1 - \xi_i, \forall x_i \in X$$

- Kernelization technique (Kernel trick)
 - We never need to compute $\phi(x_i)$ explicitly
 - Since SVM only depend on inner products, we replace $\phi(x_i) \cdot \phi(x'_i)$ with a kernel function $K(x_i, x'_i)$

Definitions

Kernel Trick

- A kernel is a function that quantifies the similarity of two observations
 - $K(x_i, x_j) = \sum_{z=1}^m x_i^z x_j^z$
 - z stands for the z th feature
- $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle, \forall x_i, x_j \in X$
 - $\langle \phi(x_i), \phi(x_j) \rangle \Rightarrow \phi(x_i) \cdot \phi(x_j)$

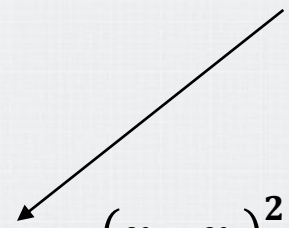
Example

Kernel Trick

- Let $\phi(x) = \begin{bmatrix} x^1 x^1 \\ x^1 x^2 \\ x^2 x^1 \\ x^2 x^2 \end{bmatrix}$

- Let $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j) = \begin{bmatrix} x_i^1 x_i^1 \\ x_i^1 x_i^2 \\ x_i^2 x_i^1 \\ x_i^2 x_i^2 \end{bmatrix} \cdot \begin{bmatrix} x_j^1 x_j^1 \\ x_j^1 x_j^2 \\ x_j^2 x_j^1 \\ x_j^2 x_j^2 \end{bmatrix} = \dots = (x_i \cdot x_j)^2$

For the sake of simplicity



- Observe that instead of mapping to 4 dimensions ($\phi(\cdot)$) and computing the inner product, we simply compute $(x_i \cdot x_j)^2$

Inference

Kernel Trick

- Assume a single sample x and S the set of support vectors ($S \subset X$)
- Linear
 - $w = \sum_{i=1}^S \alpha_i y_i x_i$
 - $f(x) = w \cdot x + b$
- Kernel
 - $w = \sum_{i=1}^S \alpha_i y_i K(x_i, x)$
 - $f(x) = w + b$

[Hands-on](#)



Practical Issues

Kernel Trick

- Algorithms that only depend on inner products between sample points
- Replacing the original inner product in the input space with positive definite kernels immediately extends algorithms such as SVMs to a linear separation in that high-dimensional space
 - Equivalently, to a non-linear separation in the input space

Popular Kernel

Kernel Trick

- Polynomial

- $K(x_i, x_j) = (1 + \sum_{k=1}^m x_i^k x_j^k)^d$
- d is positive integer – hyperparameter

- Radial

- $K(x_i, x_j) = \exp(-\gamma \sum_{k=1}^m ((x_i^k - x_j^k)^2))$
- γ is a positive constant – hyperparameter

[Hands-on](#)



Multi-class Algorithms

Motivation

Multi-class Algorithms

- In order to deal with multi-class tasks, some approaches reduce the problem to that of multiple binary classification tasks
- One-Versus-All
- One-Versus-One

One-Versus-All

Multi-class Algorithms

- This technique consists of learning k **binary classifiers**
 - One-versus-the-rest technique
- The technique seeks to discriminate one class from all the others
 - For this purpose, we need to relabel the dataset C times
 - Each time, the label from class c is 1 and the rest is -1
 - We train a binary classifier $f_c(.)$ for each combination
- Prediction
 - $\hat{y} = \operatorname{argmax}_{c \in C} f_c(x)$

[Hands-on](#)



One-Versus-One

Multi-class Algorithms

- This technique consists of using the training data to learn (independently) a binary classifier for each pair of distinct classes
 - Therefore, it requires training $\binom{k}{2} = k(k-1)/2$ binary classifiers
- Prediction
 - Majority voting
 - $\hat{y} = \operatorname{argmax}_{c \in C} \sum_{i=1}^k I(\hat{y}_i = c)$
 - I is the indicator function

$f_1(.)$	Class 1 vs. Class 2
$f_2(.)$	Class 1 vs. Class 3
$f_3(.)$	Class 2 vs. Class 3

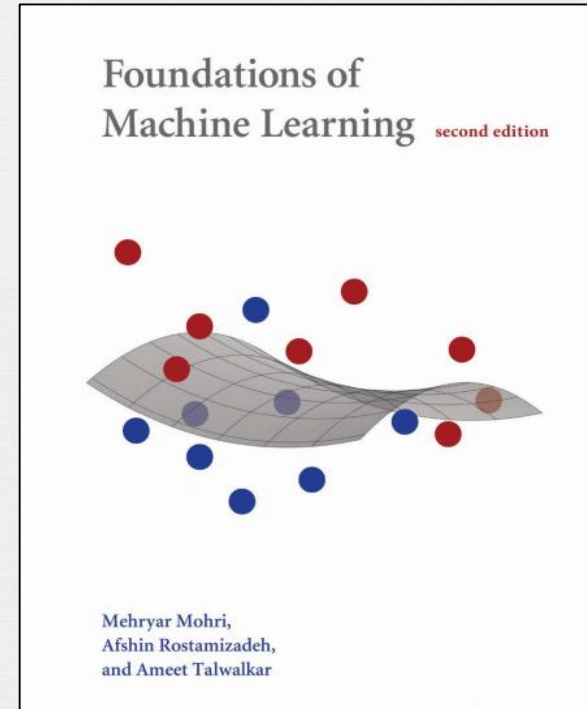
Hands-on



Bibliography

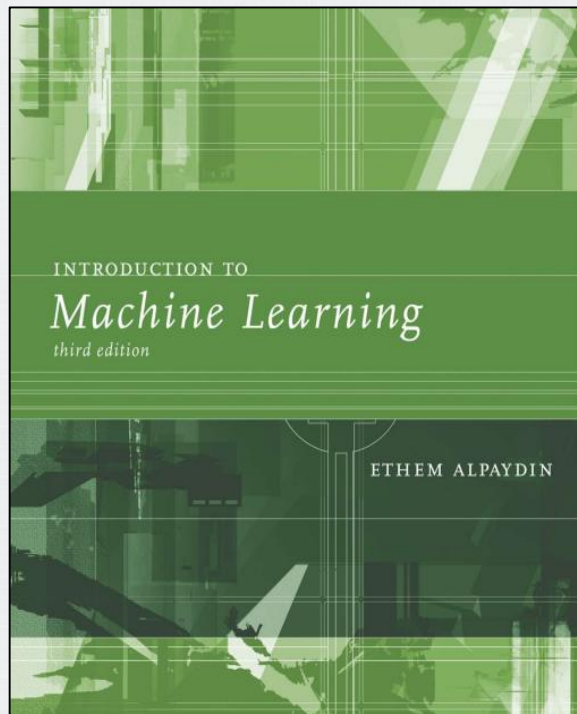
Bibliography

- Foundations of Machine Learning
 - Chapter 5 Support Vector Machines
 - Chapter 9
 - 9.4 Aggregated multi-class algorithms



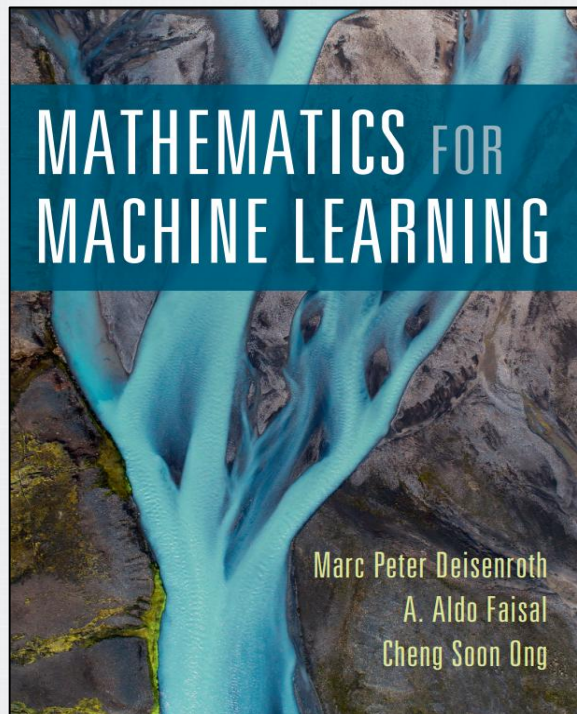
Bibliography

- INTRODUCTION TO Machine Learning
 - Chapter 13 Kernel Machines



Bibliography

- MATHEMATICS FOR MACHINE LEARNING
 - Chapter 12
Classification with Support Vector Machines
 - Chapter 7
 - 7.2 Constrained Optimization and Lagrange Multipliers



Bibliography

- PATTERN RECOGNITION AND MACHINE LEARNING
 - Chapter 7
 - 7.1.3 Multiclass SVMs

